

COMPLETE RAG ARCHITECTURE GUIDE FOR BEGINNERS

Retrieval-Augmented Generation — From Fundamentals to Practical Implementation

Author: RAG Learning Project | **Level:** Absolute Beginner → Intermediate

Technology Stack: LangChain 1.x • Ollama • ChromaDB • Python 3.10+

Covers: Document Ingestion • Chunking • Embeddings • Vector Stores • Semantic Retrieval • LLM Prompt Chains

■ What this Guide Gives You:

This comprehensive manual demystifies RAG step-by-step. You will understand why traditional LLMs hallucinate, how chunking preserves semantic meaning, how mathematical embedding vectors represent concepts, how ChromaDB executes similarity search, and how LangChain coordinates the full retrieval pipeline with local open-source models using Ollama.

Author: RAG Learning Project

Level: Absolute Beginner → Intermediate

Last Updated: 2026

"Give a man a fish and you feed him for a day. Teach a man to fish and you feed him for a lifetime."

RAG teaches the AI WHERE to fish for information, not just how to guess.

■ TABLE OF CONTENTS

- 1. [What is RAG and Why Does it Exist?](#)
- 2. [The Big Picture: RAG Architecture Overview](#)
- 3. [Stage 1: Document Loading](#)
- 4. [Stage 2: Chunking — Splitting Text into Pieces](#)
- 5. [Stage 3: Embeddings — Converting Text to Numbers](#)
- 6. [Stage 4: Vector Stores — The Smart Database](#)
- 7. [Stage 5: Retrieval — Finding What Matters](#)
- 8. [Stage 6: The LLM — The Brain that Answers](#)
- 9. [Stage 7: The Prompt Template — Instructing the LLM](#)
- 10. [Putting It All Together: The RAG Chain](#)
- 11. [LangChain: The Framework That Connects Everything](#)
- 12. [Ollama: Running LLMs Locally](#)
- 13. [Common Mistakes and How to Avoid Them](#)
- 14. [Tuning Your RAG System](#)
- 15. [Glossary: All the Key Terms Explained](#)
- 16. [What to Learn Next](#)

1. What is RAG and Why Does it Exist?

The Problem with Regular LLMs

Imagine you hired the smartest professor in the world. He has read every book and research paper published until 2023. You ask him a question and he answers brilliantly — but there's a problem:

- 1. **He doesn't know anything after his training cutoff** (e.g., he doesn't know about your company's latest quarterly report)
- 2. **He sometimes makes things up** (called "hallucination")
- 3. **He can't reference your private documents** (your internal manuals, your company data)

This is exactly the problem with LLMs (Large Language Models) like ChatGPT, Llama, or Mistral.

The Solution: RAG

RAG = Retrieval-Augmented Generation

Instead of asking the professor to answer from memory, you give him the **relevant pages of the right book** before asking the question. Now he reads those pages and gives you an answer based on what he just read — not from memory.

WITHOUT RAG:
User: "What is our company's refund policy?"
LLM: *makes something up* → WRONG ■

WITH RAG:
User: "What is our company's refund policy?"
System: *searches company documents* → finds the refund policy page
LLM: *reads the page* → gives accurate answer ■

When Should You Use RAG?

Use Case	RAG Needed?
Chat with your own documents	■ Yes
Company knowledge base	■ Yes
Q&A on a specific book	■ Yes (what we built!)
Customer support bot	■ Yes
General conversation	■ No
Creative writing	■ No
Code generation	Sometimes

RAG vs. Fine-Tuning: What's the Difference?

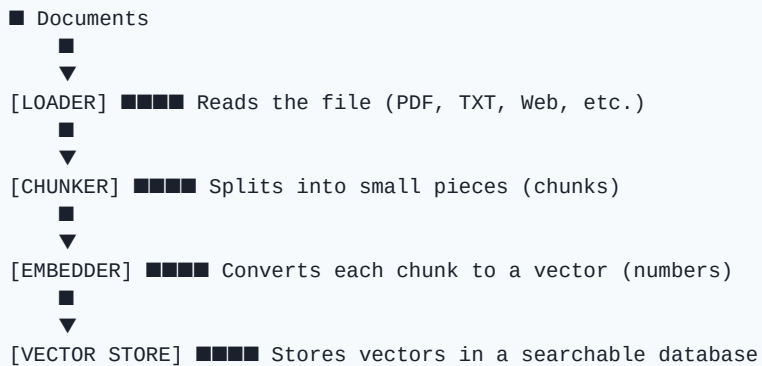
Aspect	RAG	Fine-Tuning
How it works	Give context at query time	Train the model on new data
Cost	Cheap	Very expensive (GPU hours)
Update knowledge	Just update documents	Retrain the model
Privacy	High (docs stay local)	Medium
Best for	Specific Q&A, documents	Changing behavior/style

Rule of thumb: If you want the LLM to **know new facts** → use RAG. If you want the LLM to **behave differently** → use fine-tuning.

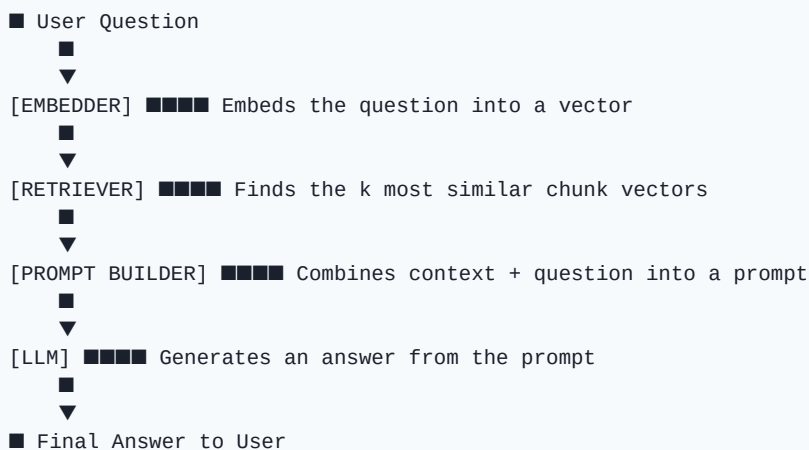
2. The Big Picture: RAG Architecture Overview

RAG has two phases:

Phase 1: Indexing (done once, offline)



Phase 2: Querying (happens every time user asks a question)



3. Stage 1: Document Loading

What is a Document Loader?

A document loader reads your source file and converts it into a format LangChain understands: **Document** objects.

A **Document** has two parts:

```

Document(
    page_content="The actual text from the file...",
    metadata={"source": "my_book.pdf", "page": 1}
)
  
```

Common Loaders in LangChain

Loader	File Type	Import
--------	-----------	--------

TextLoader	.txt files	<code>langchaincommunity.documentloaders</code>
PyPDFLoader	.pdf files	<code>langchaincommunity.documentloaders</code>
WebBaseLoader	Websites	<code>langchaincommunity.documentloaders</code>
CSVLoader	.csv files	<code>langchaincommunity.documentloaders</code>
DirectoryLoader	Whole folders	<code>langchaincommunity.documentloaders</code>
UnstructuredWordDocumentLoader	.docx files	<code>langchaincommunity.documentloaders</code>

Code Example:

```
# Load a PDF
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("my_book.pdf")
documents = loader.load()

print(f"Loaded {len(documents)} pages")
print(documents[0].page_content[:200]) # First 200 chars of page 1
print(documents[0].metadata)           # {'source': 'my_book.pdf', 'page': 0}
```

4. Stage 2: Chunking — Splitting Text into Pieces

Why Do We Chunk?

Think of chunking like organizing a library:

- You don't store an entire book on one shelf label — you organize by chapter, topic, page
- When someone asks a specific question, you find the RIGHT chapter, not the whole book

Three reasons to chunk:

1. LLM Context Limits: LLMs can only process a limited amount of text at once (called the "context window"). A 400-page book is way too long.

2. Embedding Accuracy: If you embed an entire book as one vector, you lose specificity. A vector for "chapter 3 about morning routines" is much more useful than a vector for the whole book.

3. Relevance: You want to return only the RELEVANT parts, not dump the whole document into every prompt.

How Chunking Works Visually:

ORIGINAL TEXT (very long):

"The mind is the master. Thought creates character. Character shapes destiny.
A man cannot improve his circumstances without first improving his thoughts.
The body follows the mind in all things. Health comes from pure thoughts.
Purpose gives direction to all effort. Without aim, energy is wasted..."

AFTER CHUNKING (chunk_size=100, chunk_overlap=20):

Chunk 1: "The mind is the master. Thought creates character. Character shapes destiny.
A man cannot improve his circumstances without first"

Chunk 2: "without first improving his thoughts. The body follows the mind in all things.
Health comes from pure thoughts. Purpose gives"

Chunk 3: "Purpose gives direction to all effort. Without aim, energy is wasted..."

Notice how Chunk 1 and Chunk 2 **overlap** — this prevents a sentence from being cut off right in the middle and losing its meaning.

Types of Text Splitters

1. RecursiveCharacterTextSplitter (RECOMMENDED for most cases)

Tries to split on natural language boundaries in order:

- Double newlines (paragraphs) `\n\n`
- Single newlines `\n`
- Spaces
- Character by character (last resort)

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,      # max characters per chunk
    chunk_overlap=150,   # characters shared between chunks
)
```

2. CharacterTextSplitter

Simple split on a specific character (e.g., `\n\n`):

```
from langchain_text_splitters import CharacterTextSplitter

splitter = CharacterTextSplitter(
    separator="\n\n",
    chunk_size=800,
    chunk_overlap=100,
)
```

3. TokenTextSplitter

Splits by token count (more precise for LLM context limits):

```
from langchain_text_splitters import TokenTextSplitter

splitter = TokenTextSplitter(
    chunk_size=256, # tokens, not characters
    chunk_overlap=32
)
```

4. MarkdownHeaderTextSplitter

Splits markdown by heading structure (great for documentation):

```
from langchain_text_splitters import MarkdownHeaderTextSplitter

headers_to_split_on = [
    ("#", "Header 1"),
    ("##", "Header 2"),
    ("###", "Header 3"),
]

splitter = MarkdownHeaderTextSplitter(headers_to_split_on)
```

Choosing Chunk Size: The Trade-off

Chunk Size	Pros	Cons	Best For
Small (200-400 chars)	Very precise retrieval	May lose context	FAQs, facts
Medium (500-1000 chars)	Balanced	Balanced	Most use cases ■
Large (1000-2000 chars)	More context per chunk	Less precise retrieval	Long-form answers

General rule: Start with **chunksize=800**, **chunkoverlap=150** and adjust based on your results.

5. Stage 3: Embeddings — Converting Text to Numbers

What is an Embedding?

An **embedding** is a mathematical representation of text — a list of numbers (called a **vector**) that captures the *meaning* of the text.

Think of it like GPS coordinates for meaning:

- "Paris" → coordinates (lat, lon) that place it on a map
- "London" → different coordinates, but nearby (both are European capitals)
- "Tokyo" → far away coordinates

Similarly:

- "I wake up at 5 AM" → embedding vector [0.12, -0.34, 0.89, ...]
- "Early morning routine" → similar vector [0.11, -0.31, 0.87, ...]
- "The cat sat on the mat" → very different vector [-0.88, 0.45, -0.12, ...]

The Key Property: Semantic Similarity

Texts with similar meaning → similar vectors → small distance between them

This is the magic that makes RAG work! When you ask a question, its embedding is compared to all chunk embeddings. The closest matches are the most relevant chunks.

What Does a Vector Look Like?

```
text = "Thought and character are one."
vector = [0.0123, -0.3456, 0.7891, 0.0023, -0.1234, ... ]
# 768 numbers for nomic-embed-text model
# 1536 numbers for OpenAI text-embedding-ada-002
# 384 numbers for all-MiniLM-L6-v2
```

The number of dimensions (768, 1536, 384) is the **embedding dimension** — more dimensions generally means better quality but requires more storage.

How Similarity is Measured: Cosine Similarity

The most common way to compare vectors is **cosine similarity**:

- Score = 1.0 → identical meaning
- Score = 0.7+ → very similar
- Score = 0.5 → somewhat related
- Score = 0.0 → completely unrelated
- Score = -1.0 → opposite meaning

```
Vector A: [1, 0, 0]
Vector B: [1, 0, 0]    Cosine similarity = 1.0 (identical!)

Vector A: [1, 0, 0]
Vector C: [0, 1, 0]    Cosine similarity = 0.0 (90° apart, unrelated)
```

Embedding Models Available in Our Setup

Model	Dimensions	Size	Speed	Quality
nomic-embed-text (Ollama)	768	274 MB	Fast	Good ■
mxbai-embed-large (Ollama)	1024	670 MB	Medium	Better
all-MiniLM-L6-v2 (HuggingFace)	384	22 MB	Very Fast	Decent
text-embedding-ada-002 (OpenAI)	1536	API	Fast	Excellent

Code Example:

```
from langchain_ollama import OllamaEmbeddings

# Initialize embedding model
embeddings = OllamaEmbeddings(model="nomic-embed-text")

# Embed a single query
question_vector = embeddings.embed_query("What is the power of thought?")
print(f"Dimensions: {len(question_vector)}") # 768

# Embed multiple documents at once
texts = ["Text one", "Text two", "Text three"]
doc_vectors = embeddings.embed_documents(texts)
print(f"Got {len(doc_vectors)} vectors")
```

6. Stage 4: Vector Stores — The Smart Database

What is a Vector Store?

A **vector store** is a specialized database designed to:

- 1. **Store** text + its embedding vector together
- 2. **Search** efficiently for vectors closest to a query vector

Regular databases search by exact match (**WHERE** name = 'John').

Vector stores search by **semantic similarity** (FIND texts whose meaning is closest to this query).

How a Vector Store Works Internally:

STORE PHASE (indexing):

ID	Text (original)	Vector
001	"Man is made by his thoughts"	[0.12, -0.34, ...]
002	"The body obeys the mind"	[0.88, 0.21, ...]
003	"Purpose guides all action"	[0.05, -0.67, ...]
...	...	...

SEARCH PHASE (querying):

Query: "How does thought affect the body?"

Query vector: [0.77, 0.18, ...]

- Compare to all stored vectors
- Find k closest (by cosine similarity)
- Return: chunk 002 (most relevant), chunk 001, chunk 003

Vector Store Options

Vector Store	Type	Best For	Persistence
ChromaDB	Local	Learning, prototyping	Disk 🟦

FAISS	Local	High performance, large datasets	RAM/Disk
Pinecone	Cloud	Production, scale	Cloud
Weaviate	Cloud/Self-hosted	Enterprise	Cloud
Qdrant	Cloud/Self-hosted	Production	Cloud/Disk
pgvector	PostgreSQL extension	Already using PostgreSQL	Disk

ChromaDB (What We Use)

ChromaDB is perfect for learning because:

- Works completely in-memory or on disk (no external server)
- Simple Python API
- Persistent (survives notebook restarts)
- Fast enough for thousands of chunks

```
from langchain_community.vectorstores import Chroma # Or: from langchain_chroma import Chroma
from langchain_ollama import OllamaEmbeddings

# Create & persist vector store
vectorstore = Chroma.from_documents(
    documents=chunks,          # Your text chunks
    embedding=OllamaEmbeddings(...), # Embedding model
    persist_directory="./chroma_db", # Where to save on disk
    collection_name="my_knowledge_base"
)

# Load existing (second run – no re-embedding!)
vectorstore = Chroma(
    persist_directory="./chroma_db",
    embedding_function=OllamaEmbeddings(...),
    collection_name="my_knowledge_base"
)
```

7. Stage 5: Retrieval — Finding What Matters

What is a Retriever?

A **retriever** wraps the vector store and provides a standard interface for finding relevant documents. It answers the question: *"Given this query, which chunks should I send to the LLM?"*

Types of Retrieval

1. Similarity Search (Most Common)

Returns the **k** chunks with highest cosine similarity to the query:

```
retriever = vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 3} # Return top 3 chunks
)
```

2. MMR (Maximum Marginal Relevance)

Returns relevant AND diverse chunks (avoids returning near-duplicate content):

```
retriever = vectorstore.as_retriever(
    search_type="mmr",
    search_kwargs={"k": 3, "fetch_k": 10}
    # Fetches 10, then picks 3 that are most relevant AND diverse
)
```

3. Similarity with Score Threshold

Only returns chunks above a minimum similarity score:

```
retriever = vectorstore.as_retriever(
    search_type="similarity_score_threshold",
    search_kwargs={"score_threshold": 0.7, "k": 5}
)
```

How Many Chunks to Retrieve? (k value)

k value	Effect
k=1	Very precise, but might miss context
k=3	Good balance (recommended to start) ■
k=5	More context, but slower and more tokens
k=10+	Lots of context; risk of irrelevant info

Rule: Start with **k=3**. If answers seem incomplete, increase to **k=5**.

8. Stage 6: The LLM — The Brain that Answers

What is an LLM?

A **Large Language Model (LLM)** is an AI trained on massive amounts of text to predict the next word. Through this training, it develops an understanding of language, facts, and reasoning.

Examples: GPT-4, Claude, Llama 3, Mistral, Gemma, Phi

In RAG, the LLM's job is simple:

- 1. Read the retrieved context
- 2. Read the user's question
- 3. Generate a relevant, accurate answer

Why Ollama for Local LLMs?

Ollama is a tool that makes running LLMs locally as easy as running any other program:

Cloud LLMs (OpenAI, Anthropic):	Local LLMs (Ollama):
- Need API key	- No API key
- Pay per token	- Free forever
- Data sent to their servers	- Data stays on your machine
- Requires internet	- Works offline
- Always available	- Depends on your hardware

Model Comparison

Model	RAM Needed	Speed	Quality	Best For
llama3.2:1b	1 GB	Very Fast	Basic	Low-end hardware
llama3.2 (3B)	2 GB	Fast	Good ■	Learning
llama3.1:8b	5 GB	Medium	Very Good	Better answers
mistral (7B)	4 GB	Medium	Very Good	Balanced
llama3.1:70b	40 GB	Slow	Excellent	High-end GPU

Temperature: Controlling Creativity

```
llm = OllamaLLM(model="llama3.2", temperature=0.1)
```

Temperature	Effect	Use Case
0.0	Fully deterministic, always same answer	Factual Q&A
0.1	Near-deterministic, slightly varied	RAG (recommended) ■
0.5	Balanced creativity	General chat
1.0	Creative, unpredictable	Creative writing
1.5+	Very random, often incoherent	Experimental

For RAG: Use temperature 0.0-0.2 — you want factual answers, not creative ones!

9. Stage 7: The Prompt Template — Instructing the LLM

What is a Prompt Template?

A **prompt template** is a pre-written structure that combines:

- The retrieved context
- The user's question
- Instructions to the LLM

This is the "glue" between retrieval and generation. Without a good prompt template, even perfect retrieval produces bad answers.

A Basic RAG Prompt Template:

```
You are a helpful assistant. Answer the question using ONLY the context below.
If the answer is not in the context, say "I don't know."

CONTEXT:
{context}

QUESTION: {question}

ANSWER:
```

The `{context}` and `{question}` are **placeholders** that get filled in at query time.

Why "Use ONLY the context"?

This is critical! Without this instruction:

- The LLM combines its training knowledge + context → might contradict your documents
- Hard to trace WHERE the answer came from
- Risk of hallucination

With "Use ONLY the context":

- Answers are grounded in your documents
- If info isn't in docs → LLM says "I don't know" (honest!)
- Fully auditable

Prompt Engineering Tips for RAG:

```
# Good RAG prompt structure:
GOOD_PROMPT = """
You are an expert assistant answering questions about [TOPIC].

Use ONLY the provided context to answer. Do not use prior knowledge.
If the answer is not in the context, say: "The document does not contain this information."
Keep answers concise but complete.
Cite which part of the context supports your answer.

CONTEXT FROM DOCUMENT:
{context}

USER QUESTION: {question}

ANSWER: """

# Bad prompt (no grounding):
BAD_PROMPT = """
Answer this question: {question}
Here's some info: {context}
"""
```

10. Putting It All Together: The RAG Chain

LangChain Expression Language (LCEL)

LangChain uses a pipe `|` operator to chain steps together (like Linux pipes):

```
rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)

# Usage:
answer = rag_chain.invoke("What is the power of thought?")
```

Step-by-Step What Happens:

1. User calls: `rag_chain.invoke("What is the power of thought?")`
2. RETRIEVER receives "What is the power of thought?"
 - Embeds question: `[0.23, -0.41, 0.77, ...]`
 - Searches ChromaDB for 3 nearest vectors
 - Returns 3 Document objects
3. `format_docs` receives the 3 Documents
 - Joins their `page_content` with `"\n\n--\n\n"`
 - Returns one long string (the context)
4. `RunnablePassthrough()` passes the question unchanged
5. `prompt` receives `{context: "...", question: "..."}`
 - Fills in the template placeholders
 - Returns the complete filled prompt
6. `llm` receives the filled prompt
 - Generates text response
 - Returns raw LLM output string
7. `StrOutputParser()` cleans the output
 - Returns the final clean answer string
8. User receives the answer!

11. LangChain: The Framework That Connects Everything

What is LangChain?

LangChain is a Python (and JavaScript) framework that provides:

- Standard interfaces for LLMs, embeddings, vector stores
- Pre-built chains for common patterns (RAG, chat with history, etc.)
- Easy switching between providers (swap OpenAI for Ollama in one line)

Key LangChain Components

LangChain Ecosystem:	
■■■ langchain	← Core abstractions
■■■ langchain-community	← Integrations (loaders, vectorstores)
■■■ langchain-ollama	← Ollama-specific (LLM + embeddings)
■■■ langchain-openai	← OpenAI integration (GPT-4, embeddings)
■■■ langgraph	← For complex agents and workflows

Why LangChain Instead of Coding from Scratch?

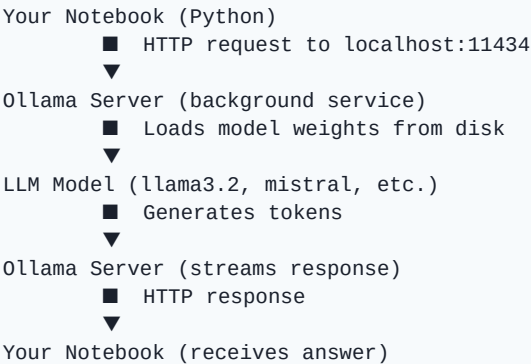
Without LangChain, you'd need to:

- 1. Manually call Ollama HTTP API
- 2. Write your own chunking logic
- 3. Implement cosine similarity search
- 4. Build the prompt manually
- 5. Parse the LLM response

With LangChain, each step is one line of code. You focus on the problem, not the plumbing.

12. Ollama: Running LLMs Locally

How Ollama Works



Ollama API Endpoints

Endpoint	What it does
POST /api/generate	Generate text from a prompt
POST /api/embeddings	Get vector embedding of text
GET /api/tags	List downloaded models
POST /api/pull	Download a new model

The Model Lifecycle in Ollama

1. `ollama pull llama3.2` → Downloads model (once, ~2GB)
2. `ollama run llama3.2` → Loads model into RAM
3. LangChain sends requests → Model processes them
4. Model stays in RAM → Fast for subsequent requests
5. `ollama stop llama3.2` → Unloads model, frees RAM

13. Common Mistakes and How to Avoid Them

Mistake 1: Chunk Size Too Large

- Problem: `chunk_size=5000` (too big)
 - Few chunks created
 - Each retrieval brings back huge blocks of irrelevant text
 - LLM gets confused, answers become generic
- Fix: `chunk_size=500-1000` for most documents

Mistake 2: No Chunk Overlap

- Problem: `chunk_overlap=0`
 - A sentence like "The main principle is [end of chunk] that thought..."
 - The key idea is split across two chunks
 - Neither chunk makes sense alone
- Fix: `chunk_overlap = 10-20% of chunk_size`
If `chunk_size=800`, use `chunk_overlap=100-160`

Mistake 3: k Too High

- Problem: `k=20` (retrieving 20 chunks)
 - Retrieval returns many irrelevant chunks
 - LLM gets confused by too much context
 - Slower response
- Fix: Start with `k=3`, increase to `k=5` if needed

Mistake 4: Bad Prompt Template

- Problem: No instruction to "use only the context"
 - LLM mixes training knowledge with retrieved context
 - Hallucination occurs
- Fix: Always include "Answer ONLY using the provided context"

Mistake 5: Not Persisting the Vector Store

- Problem: Creating in-memory ChromaDB
 - Every notebook restart requires full re-embedding
 - 3-10 minutes wasted each time
- Fix: Always set `persist_directory="./chroma_db"`
On second run, load with: `Chroma(persist_directory=...)`

Mistake 6: Embedding Different Models for Index vs Query

- Problem: Index with Model A, Query with Model B
 - The vectors are in completely different spaces
 - Similarity search returns garbage
- Fix: ALWAYS use the SAME embedding model for both indexing and querying

14. Tuning Your RAG System

When Answers Are Wrong or Incomplete:

Diagnosis checklist:

1. Check retrieval first: print the retrieved chunks — are they relevant?
2. If chunks aren't relevant → reduce `chunk_size`, increase `k`
3. If chunks are relevant but answer is wrong → improve the prompt
4. If chunks are relevant and prompt is good → upgrade the LLM

RAG Tuning Parameters

```
# Experiment with these settings:

# Chunking
chunk_size = 800          # Try: 400, 600, 800, 1200
chunk_overlap = 150       # Try: 50, 100, 150, 200

# Retrieval
k = 3                     # Try: 2, 3, 5, 7
search_type = "similarity" # Try: "mmr" for diverse results

# LLM
temperature = 0.1        # Try: 0.0, 0.1, 0.3
model = "llama3.2"        # Try: "mistral", "llama3.1:8b"
```

Evaluating RAG Quality

Ask your RAG system these types of questions and rate the answers:

Question Type	Good Answer =
---------------	---------------

Direct fact lookup	Exact answer from document
Concept explanation	Clear explanation matching document
Comparison	Lists and compares elements from document
Summary	Accurate summary of document content
Out-of-scope	"I don't have that information" (NOT a hallucinated answer)

15. Glossary: All the Key Terms Explained

Term	Simple Explanation
RAG	Retrieval-Augmented Generation — giving the LLM relevant documents before asking it to answer
LLM	Large Language Model — an AI trained to generate and understand text (e.g., Llama, GPT-4)
Chunking	Splitting a long document into smaller, manageable pieces
Chunk	A single piece of split text, typically 500-1000 characters
Chunk Size	Maximum number of characters (or tokens) in one chunk
Chunk Overlap	How many characters are shared between adjacent chunks
Embedding	A list of numbers (vector) that represents the meaning of text
Vector	A list of floating-point numbers representing a point in mathematical space
Embedding Model	A neural network that converts text into embedding vectors
Embedding Dimension	The length of the vector (e.g., 768 means 768 numbers)
Cosine Similarity	A measure of how similar two vectors are (0.0 to 1.0)
Vector Store	A database designed to store and search embedding vectors
ChromaDB	An open-source vector database we use for storage
Retriever	A component that finds the most relevant chunks for a given query
k	Number of chunks to retrieve per query
MMR	Maximum Marginal Relevance — retrieval that balances relevance + diversity
Prompt Template	A pre-written text structure with placeholders for context and question
Context Window	The maximum amount of text an LLM can process at once
Hallucination	When an LLM generates false information confidently
Grounding	Ensuring LLM answers are based on provided documents, not training memory

Ollama	A tool for running LLMs locally on your machine
LangChain	A Python framework for building LLM-powered applications
LCEL	LangChain Expression Language — the `
Temperature	Controls LLM creativity (0 = deterministic, 1 = creative)
Token	A word or word-fragment that LLMs process (roughly 0.75 words per token)
Fine-tuning	Training an LLM on new data to change its behavior (vs. RAG which uses context)
Semantic Search	Searching by meaning (not exact keywords) using embeddings
Inference	The process of an LLM generating an answer from a prompt
Persist	Saving data to disk so it survives program restarts
Collection	A named group of vectors in ChromaDB (like a table in a database)

16. What to Learn Next

You've mastered the fundamentals! Here's a learning roadmap:

Beginner → Intermediate

- ☐ Try loading a PDF instead of text (**PyPDFLoader**)
- ☐ Experiment with different chunk sizes and observe the effect
- ☐ Try **search_type="mmr"** in the retriever
- ☐ Try a different Ollama model (Mistral, Gemma)
- ☐ Add source citation to answers

Intermediate → Advanced

- ☐ **Multi-document RAG**: Load an entire folder of PDFs
- ☐ **Hybrid Search**: Combine semantic search + keyword search (BM25)
- ☐ **Re-ranking**: Use a cross-encoder to re-rank retrieved results
- ☐ **Query transformation**: Rewrite the user's question for better retrieval
- ☐ **HyDE (Hypothetical Document Embeddings)**: Generate a hypothetical answer first, embed it, then search
- ☐ **Conversation-aware RAG**: Use chat history to reformulate queries

Advanced → Expert

- ☐ **Agentic RAG**: LLM decides WHEN to retrieve and WHAT to search for
- ☐ **Graph RAG**: Build knowledge graphs from documents
- ☐ **RAG Evaluation**: Use RAGAS framework to measure precision/recall
- ☐ **Production deployment**: FastAPI backend + React frontend

